

LEAP: Linear Equations for Classifier Accuracy Prediction under Prior Probability Shift

Lorenzo Volpi^{1*}, Alejandro Moreo¹ and Fabrizio Sebastiani¹

^{1*}Istituto di Scienza e Tecnologie dell’Informazione, Consiglio Nazionale delle Ricerche, Via Giuseppe Moruzzi 1, 56124, Pisa, Italy.

*Corresponding author(s). E-mail(s): lorenzo.volpi@isti.cnr.it;
Contributing authors: alejandro.moreo@isti.cnr.it;
fabrizio.sebastiani@isti.cnr.it;

Abstract

The standard technique for predicting the accuracy that a classifier will have on unseen data (*classifier accuracy prediction* – CAP) is cross-validation (CV). However, CV relies on the assumption that the training data and the test data are sampled from the same distribution, an assumption that is often violated in many real-world scenarios. When such violations occur (i.e., in the presence of *dataset shift*), the estimates returned by CV are unreliable. The contribution of this paper is three-fold. First, we propose a CAP method specifically designed to work under *prior probability shift* (PPS), an instance of dataset shift in which the training and test distributions are characterized by different class priors. This method estimates the $\mathbf{n}^2$ entries of the contingency table of the test data (thus allowing to estimate the value of any specific evaluation measure) by solving a system of $\mathbf{n}^2$ independent linear equations, with $\mathbf{n}$ the number of classes. Second, we show that the equations that the cells of the contingency table must satisfy are actually more than $\mathbf{n}^2$, which gives rise to an overconstrained problem, and present a family of methods each based on a different selection of $\mathbf{n}^2$ such equations. Third, we observe that, since a key step of the above methods involves predicting the class priors of the test data, one can exploit intuitions from the field of class prior estimation (a.k.a. “quantification”). Our experiments show that, when combined with state-of-the-art quantification techniques, under PPS our methods tend to outperform existing CAP methods.

Keywords: Classifier Accuracy Prediction, Prior Probability Shift, Label Shift, Quantification

1 Introduction

In machine learning, estimating the accuracy that a classifier will have on unseen data (*classifier accuracy prediction* – CAP) is a fundamental step towards ensuring that the classifier we deploy is trustworthy. The accuracy of a classifier (where “accuracy” is broadly understood as any effectiveness metric – e.g., vanilla accuracy, or F_1 , or other) is typically estimated by means of cross-validation (CV) on the training data. However, CV relies on the so-called IID assumption, according to which the training data and the test data are sampled from the same distribution. Such an assumption is often violated in many real-world scenarios. When this happens, the use of CV often leads to unreliable estimates of classifier accuracy, ultimately compromising the choice of the classifier to be deployed and/or misleading its optimization process. When the IID assumption is not verified, i.e., when *dataset shift* [1] is present, CAP is thus an open problem.

One specific type of dataset shift is *prior probability shift* (PPS), defined as the type of dataset shift in which (a) the class priors of the training distribution are different from those of the test distribution, while (b) the class-conditional distribution of the covariates is invariant across the training data and the test data. In this paper we propose LEAP (for “Linear Equations for Classifier Accuracy Prediction”), a simple yet effective family of methods for CAP which are specifically tailored to work under PPS.

The contribution of this paper is three-fold. First, we argue that, given that the vast majority of classifier accuracy measures are computed on a contingency table that relates the true labels to the predicted labels, CAP can be approached by treating the entries of this table as unknowns that require estimation. Using these estimates, the accuracy of the classifier on the test data can be estimated too, according to one or more accuracy measures at the same time; this is in contrast to previously proposed methods, that estimate accuracy according to a single measure and thus need retraining if the estimate of a different measure is needed. In a classification problem with n classes, the contingency table has n^2 entries; we show that these entries must satisfy a number of constraints, we show that these constraints are linear equations, and we present a system of n^2 independent such equations, whose solution results in the prediction of the n^2 entries.

Our second contribution is based on the observation that the equations that the cells of the contingency table must satisfy are actually more than n^2 , which gives rise to an overconstrained problem. Following this observation, we present a family of methods each based on a different choice of n^2 equations that allow estimating the entries of the contingency table.

Our third contribution starts from the observation that a key step in our method involves predicting the prevalence values (i.e., the priors, or relative frequencies) of the classes in the test data. This suggests a connection with the field of *learning to quantify* [2–4], the supervised learning task of predicting the distribution of the class labels in the test data. We thus propose CAP methods whereby, for the goal of solving

the above-mentioned n^2 equations, the class priors of the test data are estimated by “quantification” algorithms tailored to scenarios affected by PPS.¹

The rest of the paper is structured as follows. Section 2 reviews the related literature on classifier accuracy prediction. In Section 3 we define the notation and the concepts we use in the rest of the paper. Section 4 is devoted to explaining our novel methods, the set of equations on which they are based, and their connections with quantification learning. We present our experimental results in Section 5, while Section 6 wraps up and discusses potential ideas for future work.

2 Related Work

A few methods for CAP under dataset shift have emerged in recent years [6–11].

The *Average Thresholded Confidence* (ATC) method of Garg et al. [6] learns a threshold t on the confidence (viewed in terms of posterior probabilities) that the classifier has in a classification decision, such that datapoints classified with confidence higher than t are assumed to be correctly classified. The authors suggest two functions for computing this score, based, respectively, on maximum confidence and negative entropy. Threshold t can be learned on the validation set as the value such that the portion of datapoints classified with confidence higher than it corresponds to the portion of correctly classified datapoints.

The *Difference of Confidence* (DoC) method of Guillory et al. [7] also leverages the confidence a model shows in its predictions, since it uses the confidence scores returned by the model to train a regressor that estimates the accuracy of the model on the test set. In addition to the validation set V , the method assumes the existence of several validation samples V_i exhibiting some type of dataset shift with respect to V (and ideally resembling the type of dataset shift to be faced in deployment conditions). For every validation sample V_i , the average max confidence of the classifier (C^{V_i}) is computed. The scores used to train the regressor are computed as the “Difference of Confidences” (DoC – hence the name of the method) between V and each V_i , i.e., as $\text{DoC}(V, V_i) = C^V - C^{V_i}$. A regressor is thus trained on these scores to predict the difference in accuracy that h exhibits on V and U . The accuracy of h on U can be computed straightforwardly from the predicted score $\text{DoC}(V, U)$ and the (known) accuracy of h on V .

The *Reverse Classification Accuracy* (RCA) method of Elsahar and Gallé [8] is based on a “back-and-forth” process in which the classifier h to be evaluated is compared with a classifier h' trained on the (pseudo-)labels generated by h on U ; both classifiers are then applied to validation set V , and their difference in accuracy (the “RCA score”) is input to a regressor that returns the estimated accuracy of h on U . In order to train this regressor, a number of validation samples V_i must be available, where a training example for training the regressor is composed of (a) the difference

¹This paper is an extended version of [5], where we originally discuss the LEAP_{ACC} and $\text{LEAP}_{\text{KDEy}}$ methods of Sections 4.3 and 4.4, respectively. In the present paper we generalize LEAP to a family of methods, of which LEAP_{ACC} and $\text{LEAP}_{\text{KDEy}}$ are specific instances, and propose two additional methods, the S-LEAP and O-LEAP methods of Sections 4.2 and 4.5, respectively. Additionally, while in [5] the experimental part only dealt with the binary setting, we now carry out extensive experimentation of all or methods for the multiclass case too.

in the accuracy values obtained on V by classifier h and a classifier h' trained on V_i (the covariate), and (b) the accuracy of h on V_i (the label).

Mandoline [9] belongs instead to the family of “importance-weighting methods” [12]. Methods in this family try to estimate the accuracy of h on U using estimates $\hat{r}(\mathbf{x})$ of the true *density ratio* $r(\mathbf{x}) = \frac{q(\mathbf{x})}{p(\mathbf{x})}$, with p and q the density functions of the train and test probability distributions P and Q , respectively. Mandoline relies on user-defined so-called “slicing” functions to create common representations and compare training data with test data.

Chen et al. [10] exploit model agreement in an iterative framework, training an ensemble of auxiliary models for each iteration and using their agreement ratio to estimate the accuracy of the original model.

Generalisation Disagreement Equality (GDE) [11] is instead based on the observation that the degree of disagreement on in-distribution (i.e., IID) data between identical neural architectures that have been initialised differently correlates strongly with their accuracy on (non-IID) test data.

The methods we present in this paper are very different from all the methods we have mentioned above since, unlike the latter, they are *not* based on estimating an accuracy measure directly, but are based on estimating (by solving a system of linear equations) the cells of the contingency table, from which any measure can in turn be computed. In cases where more than one evaluation measure needs to be estimated, the methods presented above require retraining, while our methods allow estimating several evaluation measures at once.

3 Background

3.1 Notation

Throughout this paper we use the following notation. By $\mathcal{X} = \mathbb{R}^d$ and $\mathcal{Y} = \{\omega_1, \dots, \omega_n\}$ we denote the input space (vectors of covariates) and the output space (classes), respectively; when dealing with binary problems we will use the shorthand $\mathcal{Y} = \{\oplus, \ominus\}$. By $D = \{(\mathbf{x}_l, y_l)\}_{l=1}^m$ we denote a generic labelled set consisting of pairs $\mathbf{x}_l \in \mathcal{X}$ and $y_l \in \mathcal{Y}$, that we will use for training or testing our models.

By $h : \mathcal{X} \rightarrow \mathcal{Y}$ we denote a (crisp) classifier mapping vectors of covariates into classes. By $h : \mathcal{X} \rightarrow \Delta^{n-1}$ we instead denote a probabilistic classifier mapping vectors of covariates into vectors of posterior probabilities lying in the probability simplex $\Delta^{n-1} = \{(\delta_1, \dots, \delta_n) : \delta_i \geq 0, \sum_{i=1}^n \delta_i = 1\}$. From a probabilistic classifier one can easily obtain a crisp classifier by returning, for a given datapoint, the class corresponding to the largest posterior probability.

3.2 Prior Probability Shift

Dataset shift is defined as the situation in which the training set is drawn from a distribution P (hereafter: “the training distribution”) and the test set is drawn from another distribution Q (“the test distribution”) such that $P(X, Y) \neq Q(X, Y)$. *Prior probability shift* (PPS – sometimes called *target shift* [13] or *label shift* [14]) is a type of dataset shift (to be found in *anti-causal learning* [15], i.e., the class of learning

problems in which Y represents the phenomenon to be predicted and X represents *symptoms* of this phenomenon) defined as the case in which

$$\begin{aligned} P(Y) &\neq Q(Y) \\ P(X|Y) &= Q(X|Y) \end{aligned} \tag{1}$$

i.e., the case in which the prevalence values of the classes can change between the training distribution and the test distribution but the class-conditional distribution of the covariates does not change. If $Z = f(X)$, with f an X -measurable function, from $P(X|Y) = Q(X|Y)$ it follows (see [16]) that $P(Z|Y) = Q(Z|Y)$. In particular, if we take $f = h$, from $P(X|Y) = Q(X|Y)$ follows that the distribution of the class-conditional predictions $\hat{Y}$ issued by the classifier is the same for the training distribution P and the test distribution Q , i.e., it holds that

$$P(\hat{Y}|Y) = Q(\hat{Y}|Y) \tag{2}$$

3.3 Problem Setting

We assume a classifier h trained on a set L of labelled datapoints, which we assume may no longer be available; we also assume a validation set V of labelled datapoints, with L and V drawn from the same distribution P . CAP consists of predicting the accuracy that our classifier h will exhibit on a given test set U (for which we assume the labels are not available) in terms of an accuracy measure A , with U drawn from a distribution Q such that P and Q are related by PPS. Note that h is assumed already trained, and that our goal is *not* to improve its performance on U but just to estimate how well it will fare on U .

4 Method

Most popular measures used for evaluating the performance of a classifier h can be computed in terms of a *contingency table* (also known as a *confusion matrix*) that relates the outcomes of the random variable Y (which denotes the true class of a datapoint) with those of the random variable $\hat{Y}$ (which denotes the class predicted by h of the datapoint). A contingency table is a matrix where entry (i, j) denotes the number of datapoints whose true class is ω_i and whose predicted class is ω_j . However, without loss of generality, from now on we will take the value of each entry to be normalized by the total number of datapoints, i.e., we will consider entry (i, j) of a contingency table as the *fraction* of datapoints whose predicted class is ω_i and whose true class is ω_j ; in other words, given that these fractions sum up to 1, we will see a contingency table as an empirical *probability distribution*.

Let us consider the case in which the contingency table is obtained by applying a classifier h to a validation set $V = \{(\mathbf{x}_l, y_l)\}_{l=1}^m$ drawn from the training distribution P . Applying h to V maps V into $V' = \{(h(\mathbf{x}_l), y_l)\}_{l=1}^m$, from which we can obtain a

contingency table $\mathbf{V}$ with entries

$$c_{ij}^V = \frac{|\{(h(\mathbf{x}_l), y_l) \in V' : h(\mathbf{x}_l) = \omega_i \wedge y_l = \omega_j\}|}{|V'|} \quad (3)$$

Note that, when $|V|$ is large enough, it holds that $c_{ij}^V \approx P(\hat{Y} = \omega_i, Y = \omega_j)$.

As noted in the introduction, if the distributions P and Q from which V and U (resp.) are drawn are related by PPS, the contingency table $\mathbf{V}$ cannot be used to reliably estimate the accuracy of h on U . Ideally, to estimate the accuracy of h on U we would like to directly estimate the entries c_{ij}^U of the contingency table $\mathbf{U}$ that results when h classifies the datapoints in U , but this is not trivial, since on U we can observe the predicted labels (by simply applying h to the datapoints of U) but not the true labels. If we had access to these true labels, any evaluation measure could be computed; for example, vanilla accuracy for a test set U can be expressed as

$$\text{Acc}(h, U) = \sum_{i=1}^n c_{ii}^U \quad (4)$$

(where the absence of denominators is due to the fact that c_{ii}^U is not a count but a fraction).

In the binary case we will indicate fractions c_{11}^V and c_{11}^U as TP^V and TP^U , respectively, where TP stands for “true positives” and the V and U subscripts indicate from which set of datapoints (the validation set V or the test set U) these fractions originate;² analogously, FP, FN, TN, will stand for “false positives”, “false negatives”, “true negatives”, respectively. We will also use the shorthands

$$\text{tpr} = \frac{\text{TP}}{\text{TP} + \text{FN}} \quad \text{fpr} = \frac{\text{FP}}{\text{FP} + \text{TN}} \quad \text{tnr} = \frac{\text{TN}}{\text{FP} + \text{TN}} \quad \text{fnr} = \frac{\text{FN}}{\text{TP} + \text{FN}} \quad (5)$$

to refer to the “true positive rate”, the “false positive rate”, the “true negative rate”, and the “false negative rate” of classifier h . We use the suffix V or U to indicate the contingency table in which the rate has been computed, e.g., tpr^V or tpr^U .

4.1 The Governing Equations of the Problem

In this section we show that the values c_{ij}^U that we would like to estimate need to satisfy a number of constraints, and that these constraints are linear equations (the *governing equations* of our problem) in which the c_{ij}^U ’s are the unknowns. Since the number of values c_{ij}^U that we want to estimate is n^2 , if we have a system of n^2 independent equations we can estimate them all by solving it. As anticipated in the introduction, the main contribution of this paper is a simple yet effective family of methods for CAP, called LEAP (for “Linear Equations for Classifier Accuracy Prediction”), that are indeed based on identifying a system of n^2 independent equations whose solution

²The term “true positives” is typically used to refer to the *number* (and not the *prevalence*) of datapoints which have correctly been predicted to belong to class $\oplus$; however, in this paper it is useful for us to always think in terms of prevalence values.

allows us to estimate the c_{ij}^U values (and, from them, any accuracy measure based on a contingency table).

In the following, we derive the governing equations of our problem (that belong to four different groups, which we will present one by one) for the general multiclass case ($n > 2$), and show how they look like in the binary case ($n = 2$). Before doing so, let us recall that $0 \leq c_{ij}^U \leq 1$ for all $i, j \in \{1, \dots, n\}$.

4.1.1 The First Group of Equations

This group actually consists of a single (trivial) equation; the entries of the contingency table are probabilities of disjoint events that cover the entire event space, and they thus must sum to one, which means that our first equation is

$$\sum_{i=1}^n \sum_{j=1}^n c_{ij}^U = 1 \quad (6)$$

In the binary case, this corresponds to equation

$$\text{TP}^U + \text{FP}^U + \text{FN}^U + \text{TN}^U = 1 \quad (7)$$

4.1.2 The Second Group of Equations

This group relies on the fact that the sum of the entries in row i of $\mathbf{U}$ must equal the (known) fraction of datapoints in U that h has assigned to class ω_i . Let us define the vector $\gamma = (\gamma_1, \dots, \gamma_n) \in \Delta^{n-1}$ of these observed fractions, where

$$\gamma_i = \frac{1}{|U|} \sum_{\mathbf{x} \in U} \mathbb{1}[h(\mathbf{x}) = \omega_i] \quad (8)$$

This adds to the system n equations of type

$$\sum_{j=1}^n c_{ij}^U = \gamma_i \quad \text{for } i \in \{1, \dots, n\} \quad (9)$$

At this point, it is worth noting that the $(n+1)$ equations deriving from Equations 6 and 9 are not independent of each other; indeed, one could pick any of them and obtain it as a linear combination of the other n . However, we put aside this problem for the moment being, and will come back to it in the next section.

In the binary case, Equations 9 amount to

$$\begin{aligned} \text{TP}^U + \text{FP}^U &= \gamma_{\oplus} \\ \text{FN}^U + \text{TN}^U &= \gamma_{\ominus} \end{aligned} \quad (10)$$

4.1.3 The Third Group of Equations

This group derives from the assumption that the type of dataset shift affecting distributions P and Q is PPS. As indicated in Section 3.2, this entails that $P(\hat{Y}|Y) = Q(\hat{Y}|Y)$ (see Equation 2), which means that the classification rates do not change across the training and test distributions. Since the validation set V is sampled from the training distribution P , this means that

$$\frac{c_{ij}^U}{\sum_{k=1}^n c_{kj}^U} = \frac{c_{ij}^V}{\sum_{k=1}^n c_{kj}^V} \quad \text{for } i, j \in \{1, \dots, n\} \quad (11)$$

If we use the shorthand

$$r_{ij}^V = \frac{c_{ij}^V}{\sum_{k=1}^n c_{kj}^V} \quad (12)$$

(where r_{ij}^V denotes the classification rates in V) we can rewrite Equation 11 as

$$c_{1j}^U \cdot r_{ij}^V + \dots + c_{ij}^U (r_{ij}^V - 1) + \dots + c_{nj}^U \cdot r_{ij}^V = 0 \quad \text{for } i, j \in \{1, \dots, n\} \quad (13)$$

Since values r_{ij}^V can be computed by classifying the datapoints in V via h , Equation 13 is actually a set of n^2 equations (one for each value r_{ij}^V) in n^2 unknowns (the c_{ij}^U values).

In the binary case, this is expressed through one equation for each classification rate (tpr, fpr, tnr, fnr). For tpr (the other three cases are analogous) we have

$$\frac{\text{TP}^U}{\text{TP}^U + \text{FN}^U} = \text{tpr}^V \quad (14)$$

which can be more conveniently written as

$$\text{FN}^U \cdot \text{tpr}^V + \text{TP}^U \cdot (\text{tpr}^V - 1) = 0 \quad (15)$$

where FN^U and TP^U are the unknowns. Note that, as for the second group of equations, the $(n^2 + 1)$ equations deriving from Equations 6 and 13 are not independent; e.g., in the binary case, one of the four equations of which Equation 15 is an example is, if taken together with Equation 7, redundant. Here too, we momentarily set aside the issue of independence and defer its discussion to later on.

4.1.4 The Fourth Group of Equations

This group relies on the fact that, in contingency table $\mathbf{U}$, the sum of the elements on column j is the true prevalence of class ω_j in the test set U . At first sight this would seem hardly useful, since the true prevalence values of the classes in U are unknown.

However, it turns out that these values can be estimated from contingency table $\mathbf{V}$ with the aid of Equation 2. This is best illustrated in the binary case by noting that

$$\begin{aligned} Q(\hat{Y} = \oplus) &= Q(\hat{Y} = \oplus | Y = \oplus) Q(Y = \oplus) + Q(\hat{Y} = \oplus | Y = \ominus) Q(Y = \ominus) \\ &= P(\hat{Y} = \oplus | Y = \oplus) Q(Y = \oplus) + P(\hat{Y} = \oplus | Y = \ominus) Q(Y = \ominus) \end{aligned} \quad (16)$$

where the first step derives from the law of total probability and the second step derives from the fact that, since we assume PPS, Equation 2 holds. By taking into account (i) the fact that $Q(Y = \ominus) = (1 - Q(Y = \oplus))$; (ii) the fact that, because of Equation 2, $P(\hat{Y} = \oplus | Y = \oplus) \approx \text{tpr}^V$ and $P(\hat{Y} = \oplus | Y = \ominus) \approx \text{fpr}^V$, and (iii) the fact that an estimate of $Q(\hat{Y} = \oplus)$ (denoted by $\gamma_{\oplus}$) can be obtained from Equation 8, we obtain that

$$Q(Y = \oplus) \approx \frac{\gamma_{\oplus} - \text{fpr}^V}{\text{tpr}^V - \text{fpr}^V} \quad (17)$$

Since $\gamma_{\oplus} = \text{FP}^U + \text{TP}^U$ and $Q(Y = \oplus) \approx \text{FN}^U + \text{TP}^U$ Equation 17 leads to equation

$$\text{FP}^U + \text{FN}^U \cdot (\text{fpr}^V - \text{tpr}^V) + \text{TP}^U \cdot (1 + \text{fpr}^V - \text{tpr}^V) = \text{fpr}^V \quad (18)$$

An analogous to Equation 17 can be formulated for $Q(Y = \ominus)$, which generates an equation analogous to Equation 18. In the multiclass case, there are n such equations. The derivation is slightly more complicated, as it entails writing Equation 16 as a system of linear equations, one per class, that involves the classification rates $P(\hat{Y} = \omega_i | Y = \omega_j)$. This problem can be written in matrix form as $\boldsymbol{\gamma} = \mathbf{R}^T \mathbf{q}$, where $\boldsymbol{\gamma} = (\gamma_1, \dots, \gamma_n) \in \Delta^{n-1}$ is the vector representing the distribution of the predicted labels, $\mathbf{R}$ is our matrix of classification rates, and $\mathbf{q} = (q_1, \dots, q_n) \in \Delta^{n-1}$ is the sought class distribution on the test data. The solution thus comes down to solving the system as $\hat{\mathbf{q}} = (\mathbf{R}^T)^{-1} \boldsymbol{\gamma}$, and then adding the n equations

$$\hat{q}_j = \sum_{i=1}^n c_{ij}^U \quad \text{for } j \in \{1, \dots, n\} \quad (19)$$

to our system.

4.1.5 Summing up ...

To sum up, by taking the first equation (Equation 6), the n equations from the second group (Equation 9), the n^2 equations from the third group (Equation 13), and the n equations from the fourth group (Equation 19), we obtain a system of exactly $(1 + n + n^2 + n) = (1 + n)^2$ equations. Since the number of unknowns is only n^2 , the problem is *overconstrained*. In the following sections we discuss different alternatives (which give rise to different CAP systems in the LEAP family) for dealing with this overabundance of constraints, including (a) two reasonable choices of n^2 equations that a linear solver can easily address, and (b) a method that takes all the equations into account.

4.2 S-LEAP: A Direct Solution Based on the PPS Assumptions

Perhaps the simplest way to address the problem is to start from the n^2 equations represented by Equation 11, i.e., the ones that derive exclusively from the fact that P and Q are related by PPS. Here, we observe that the denominators of the classification rates involved in Equation 11 turn out to be nothing else than the class priors of the validation and test sets. More formally, let $\mathbf{p} = (p_1, \dots, p_n) \in \Delta^{n-1}$ and $\mathbf{q} = (q_1, \dots, q_n) \in \Delta^{n-1}$ represent the (true) class priors in the validation and test sets, respectively, where $p_j = \sum_{i=1}^n c_{ij}^V$ and $q_j = \sum_{i=1}^n c_{ij}^U$. Equation 11 can thus be rewritten as

$$c_{ij}^U = c_{ij}^V \cdot \frac{q_j}{p_j} \quad \text{for } i, j \in \{1, \dots, n\} \quad (20)$$

which allows to directly estimate each entry of the test contingency table by simply rescaling the corresponding entry from the validation contingency table.

Of course, while vector $\mathbf{p}$ is observed, $\mathbf{q}$ remains unknown. However, in Section 4.1.4 we showed a method for estimating it, which involves solving the problem $\boldsymbol{\gamma} = \mathbf{R}^\top \mathbf{q}$. By doing so, we reduce the original problem, which consists of solving a system of n^2 linear equations, to one of solving a linear system of only n equations. We call this method S-LEAP, since it is akin to *scaling* the contingency table entries by the ratios q_j/p_j between the priors.

While this method is appealing and direct, it is not without problems. The method relies entirely and solely on the PPS assumptions, which may not hold *exactly* in practice, because of the presence of additional types of shift (e.g., covariate shift), or simply because of random sampling error. Aside from this, by relying on a single group of equations, many aspects of the problem are hopelessly lost. For instance, excluding the second group of equations may well lead to column sums deviating from the known observed counts $\boldsymbol{\gamma} = (\gamma_1, \dots, \gamma_n)$.

4.3 LEAP_{ACC}: Selecting a More Representative Subset of Equations

One way to address the limitations of the S-LEAP method is to involve all four batches of equations into the final selection of n^2 equations. The method we present here follows this strategy, by selecting from the pool (i) the (only) equation from the first group (Equation 6), (ii) $(n-1)$ equations from the second group (Equation 9), (iii) $(n-1)^2$ equations from the third group (Equation 13), and (iv) $(n-1)$ equations from the fourth group (Equations 19), thus totalling exactly n^2 equations. The rationale behind this choice is to select equations from all batches while ensuring that no redundant (i.e., linearly dependent) equations are taken from the same group. The choice of the equation to be left out from each group is arbitrary; we choose to drop the equation corresponding to the index $i = j = 1$, i.e., to the first class ω_1 .

In this case, it is interesting to see that the resulting system of four equations in the binary case is

$$\begin{pmatrix} 1 & 1 & 1 & 1 \\ 0 & 0 & 1 & 1 \\ 0 & \text{tpr}^V & 0 & (\text{tpr}^V - 1) \\ 0 & 1 & 0 & 1 \end{pmatrix} \begin{pmatrix} \text{TN}^U \\ \text{FN}^U \\ \text{FP}^U \\ \text{TP}^U \end{pmatrix} = \begin{pmatrix} 1 \\ \gamma_{\oplus} \\ 0 \\ q_{\oplus} \end{pmatrix} \quad (21)$$

which has the solution

$$\begin{aligned} \text{TN}^U &= 1 - \gamma_{\oplus} + q_{\oplus}(\text{tpr}^V - 1) \\ \text{FN}^U &= q_{\oplus}(1 - \text{tpr}^V) \\ \text{FP}^U &= \gamma_{\oplus} - q_{\oplus} \cdot \text{tpr}^V \\ \text{TP}^U &= q_{\oplus} \cdot \text{tpr}^V \end{aligned} \quad (22)$$

We can now derive the closed form of many well-known evaluation measures under PPS. For example, vanilla accuracy as computed on the test contingency table $\mathbf{U}$ (Equation 23) can be expressed in compact form with the aid of γ and $\mathbf{q}$ (Equation 24), or expanded as a function that uses only γ along with counts from the validation contingency table $\mathbf{V}$ (Equation 25), as

$$\text{Acc}(h, U) = \frac{\text{TP}^U + \text{TN}^U}{\text{TP}^U + \text{TN}^U + \text{FN}^U + \text{FP}^U} \quad (23)$$

$$= 1 + q_{\oplus}(2\text{tpr}^V - 1) - \gamma_{\oplus} \quad (24)$$

$$= \frac{(\gamma_{\oplus} \cdot (\text{TN}^V + \text{FP}^V) - \text{FP}^V)(\text{TP}^V - \text{FN}^V)}{\text{TN}^V \text{TP}^V - \text{FP}^V \text{FN}^V} + 1 - \gamma_{\oplus} \quad (25)$$

The fact that this evaluation measure can be expressed in closed form is interesting, because it means that in order to perform CAP we simply need to compute Equation 25 from the counts obtained from the contingency table of the validation set, *without actually solving a system of linear equations at prediction time*. In Appendix A, we show analogous results for the case in which we use F_1 as our evaluation measure instead of Acc.

When, in the equations above, we replace the true prior $\mathbf{q}$ with our estimate $\hat{\mathbf{q}}$ (obtained via Equation 17), we obtain an instance of LEAP that we dub LEAP_{ACC} . Note that the compact formula above (Equations 24) is generic, and can be used also in cases in which the class prevalence $\mathbf{q}$ is obtained by means of other techniques. We discuss this topic in the following section, in which the rationale behind the name LEAP_{ACC} will become evident.

4.4 $\text{LEAP}_{\text{KDEy}}$: Enhancing LEAP via Quantification

The fourth group of equations (see Section 4.1.4) requires estimating the class prevalence values q_i of the test set. The solution we have shown in Equations 16 and 17

(upon which LEAP_{ACC} relies) is actually borrowed from a well-known quantification (i.e., class prevalence estimation) method, called *Adjusted Classify & Count* (ACC) [2, 17, 18]; the method has also been called “Black-Box Shift Estimator” (BBSE) [14]. This method is of particular interest for our derivation since it only involves components (as tpr and fpr) which can be derived from the contingency table of the validation set. The implementation we use in this paper does indeed adhere to the improvements brought about by Bunse [19], which come down to better handling the cases in which the matrix $\mathbf{R}^\top$ of the classification rates is non-invertible.

However, ACC is by no means the only method available for estimating class prevalence values, nor the most sophisticated one, and there is an entire body of literature devoted to devising better ways for estimating these values [3, 4]. In this section, we propose an enhanced version of our LEAP method where more sophisticated quantification algorithms are used for solving the fourth group of equations.

More formally, a quantifier is a function $\lambda : \mathcal{S} \rightarrow \Delta^{n-1}$ mapping samples $S \in \mathcal{S}$ of instances from the input space $\mathcal{X}$ to the probability simplex, where $\lambda(S) = \hat{\mathbf{q}}$ is an estimate of the true class prior distribution $\mathbf{q}$ in S . We focus our attention to the so-called *distribution-matching* approaches to quantification, which take a permutation-invariant function Φ to represent samples S , and solve for $\mathbf{q}$ (the sought test class prevalence vector) the equation $\mathbf{t} = \mathbf{z}^\top \mathbf{q}$ where $\mathbf{t} = \Phi(U)$ is the representation of test sample U and $\mathbf{z} = [\Phi(V_1), \dots, \Phi(V_n)]$ contains the class-wise representations of validation sets $V_i = \{\mathbf{x}_k : (\mathbf{x}_k, y_k) \in V, y_k = \omega_i\}$. Specifically, we adopt the KDEy-ML variant proposed in [20] (hereafter simply called KDEy), which relies on kernel density estimation for representing the (posterior probabilities of the) samples as density functions in the probability simplex. KDEy then solves the distribution matching problem via maximum likelihood, i.e., it computes

$$\hat{\mathbf{q}} = \arg \min_{\mathbf{q} \in \Delta^{n-1}} \mathcal{D}_{\text{KL}}(\mathbf{t} \parallel \mathbf{z}^\top \mathbf{q}) \quad (26)$$

where $\mathcal{D}_{\text{KL}}$ is the well-known Kullback-Leibler divergence between the density model of the test set and the density model of a mixture parameterized by $\hat{\mathbf{q}}$. We call this extension $\text{LEAP}_{\text{KDEy}}$. (Using a similar notation, by $\text{S-LEAP}_{\text{KDEy}}$ we will denote the S-LEAP method enhanced by the use of KDEy for estimating the class prevalence values.)

Yet another fundamental difference between $\text{LEAP}_{\text{KDEy}}$ and LEAP_{ACC} lies in the fact that KDEy internally uses a surrogate classifier trained on the validation set, distinct from the target classifier h whose accuracy we aim to estimate. This seems a reasonable choice, as the tool used to measure the classifier’s accuracy should arguably be independent of the classifier itself.³

4.5 O-LEAP: Overconstrained LEAP

While the method discussed in Section 4.2 can be solved directly, the methods presented in Sections 4.3 and 4.4 require solving a system of n^2 equations. Such methods

³As the surrogate classifier, that we train via cross-validation on the validation set, we choose the implementation of Multi-Layer Perceptron (MLP) available in scikit-learn, with default hyperparameters. Preliminary experiments we have run show that this choice is a good one.

can be expressed in matrix form as $\mathbf{AX} = \mathbf{b}$, where $\mathbf{X} = (c_{11}^U, \dots, c_{nn}^U)$ represents the unknowns. However, in degenerate cases in which the PPS assumptions do not hold, or the estimated class prevalence values deviate much from the true value, the system may not be solvable via matrix inversion ($\mathbf{X} = \mathbf{A}^{-1}\mathbf{b}$), or the solution may be unfeasible (e.g., some $\hat{c}_{ij}^U$ may fall outside the $[0, 1]$ range). In cases like this, we may resort to solving the constrained problem $\hat{\mathbf{X}} = \arg \min_{\mathbf{X} \in \Delta^{n^2-1}} \|\mathbf{AX} - \mathbf{b}\|_2$, with $0 \leq c_{ij}^U \leq 1$; modern software packages solve this type of system efficiently.

That said, all the methods discussed so far sacrifice, in one way or another, some of the problem’s constraints (exactly $1 + 2n$ of them are dropped). This causes an asymmetry in the way different classes are treated in the solution.⁴ We thus introduce a new method that instead incorporates all our $1 + 2n + n^2$ constraints; we call this method O-LEAP, for “Overconstrained LEAP”, as it deals with an overconstrained system of equations ($1 + 2n + n^2$ equations for only n^2 unknowns); when we want to explicitly indicate the type of quantification method used we indicate it as a subscript (say, O-LEAP_{KDEy}). The matrix $\mathbf{A} \in \mathbb{R}^{(1+2n+n^2) \times n^2}$ that derives from the entire set of constraints is rectangular, and thus non-invertible. However, the same strategy employed for “degenerate cases” can be used here, that is, we minimize the Frobenius norm of the residual $\mathbf{AX} - \mathbf{b}$ to find a solution in the overconstrained setting, aiming to satisfy, at least approximately, all the equations simultaneously.

5 Experiments

In this section we turn to describing the experiments we have carried out in order to assess the effectiveness of the LEAP family of methods.

5.1 Experimental Setup

The effectiveness measure we use in order to assess the quality of the predictions of CAP methods is absolute error (AE). For a generic classifier accuracy measure A , AE is defined as $|A(h, U) - \hat{A}(h, U)|$, where $A(h, U)$ is the true accuracy of h on U (which in lab experiments we can determine since we know the actual labels of U), and where $\hat{A}(h, U)$ is the accuracy of h on U as predicted by the CAP method.

As our classifier accuracy measure A we here employ vanilla accuracy (Acc – see Equation 4). Aside from the fact that it is one of the most important measures of classifier effectiveness, we choose Acc for the sake of fair experimental comparison, given that some of our baselines only work for this evaluation measure.

5.2 Experimental Protocol

The experimental protocol we adopt is as follows. Given a labelled collection $D = \{(\mathbf{x}_l, y_l)\}_{l=1}^m$, we split it into a training set (70%) and a test set U (30%) via stratified sampling. We further split the training set into a set L (50%) that we use for training a classifier h , and a validation set V (50%) that we use for training a CAP method, again

⁴Indeed, note that there are no fewer than $\binom{1+2n+n^2}{n^2} = \frac{(1+2n+n^2)!}{n^2(1+2n)!}$ possible ways of dropping $(1 + 2n)$ constraints; of course, the idea of exploring them all is hopeless.

using stratification. We train and test all competing methods on the same partitions. We then draw, uniformly at random, 1000 vectors $\mathbf{v}_1, \dots, \mathbf{v}_{1000}$ of prevalence values from the unit simplex Δ^{n-1} (using the Kraemer sampling algorithm [21]), and for each vector $\mathbf{v}_i$ we extract from U a bag U_i of $|U_i| = 100$ elements such that U_i satisfies the prevalence distribution of $\mathbf{v}_i$. For each test sample U_i we ask the CAP method to predict the accuracy (Acc) of h . We then compare (via AE) the predicted score with the true one and report averaged values across the 1000 tests. Note that L and V (as well as the original U) are from the same distribution P , while the test samples U_i extracted from U are instead from a different distribution Q which is related to P via PPS.

Indeed, note that this experimental protocol clearly simulates PPS, since the distribution of the covariates X conditional on the distribution of the labels Y is the same for the training set and the U_i 's, but the distribution of the labels Y is not the same for the training set and the U_i 's (in the language of Section 3.2, our protocol simulates a situation in which $P(X|Y) = Q(X|Y)$ and $P(Y) \neq Q(Y)$, i.e., enforces the PPS assumptions).

5.3 Datasets

As our datasets, we use 29 binary datasets and 21 multiclass datasets from the UCI machine learning repository [22]. We restrict our attention to those datasets that can be directly imported through the UCI's Python API, and discard datasets with fewer than 3 features or fewer than 150 instances. In the multiclass setup, we also discard datasets with more than 20 classes. The number of instances in the binary datasets varies from a minimum of 150 (iris.2, iris.3) to a maximum of 5,473 (pageblocks.5), while the number of features varies from a minimum of 3 (haberman) to a maximum of 256 (semeion). The training sets display varying degrees of balance, ranging from datasets with only 2.1% positive instances (pageblocks.5) to almost perfectly balanced datasets (mammographic), to datasets with 77.8% positive instances (ctg.1). For the multiclass case, the number of instances varies from 1,014 instances (mhr) to 1,024,985 (poker-hand), while the number of features varies from 7 (shuttle) to 227 (molecular). The number of classes varies from a 3 (mhr, connect-4, molecular, waveform-v1, cmc, academic-success) to 15 (chess). Class balance varies from highly imbalanced datasets, with some classes representing less than 1% of instances (e.g., one of the classes in poker-hand), to perfectly balanced datasets (e.g., the classes of image-seg).

5.4 Classifier-Learning Algorithms

For generating the classifiers whose accuracy we want to test, we consider four learning algorithms, i.e., logistic regression (LR), k NN (with $k = 10$), support vector machines (SVMs – using the RBF kernel), and multilayer perceptron (MLP – with hidden sizes of 100 and 15 neurons and the ReLU activation function). In all cases, we rely on the implementations provided by scikit-learn [23], leaving the rest of the hyperparameters at their default values.

5.5 Baselines

As the baselines against which we test the performance of our methods, we use ATC (with the *maximum confidence* function) [6] and DoC [7] (see Section 2); we do not report results for Mandoline [9] and RCA [8] since the results we obtain for them are not competitive with the other methods.⁵ We also report the results of Naïve, a method that assumes the training and test data to be IID, and reports the vanilla accuracy score obtained on the contingency table generated by h on V .

For the implementation of the quantification algorithms (ACC and KDEy) we rely on the QuaPy⁶ package [24]. The code to reproduce all our experiments is available on GitHub.⁷

5.6 Results

Table 1 (for the binary case) and Table 2 (for the multiclass case) show the results we obtain in our experiments in terms of AE (lower is better) for the LR classifier; the tables for k NN, SVMs, MLP, display similar trends, and are reported in Appendix B.

Overall, our results show that O-LEAP tends to obtain, on average, the best results across all four classifiers, both in the binary case and in the multiclass case. When considering either the best result for each dataset (in boldface) or results that are not statistically significantly different from the best (indicated with †), O-LEAP tends to achieve such results most frequently than any other method, across both binary and multiclass settings, as well as across all four classifier-training methods.

Concerning the baselines, our results clearly demonstrate the superiority of DoC with respect to the other baselines, particularly in the multiclass setting, where DoC almost always achieves the best result among the baselines.

The superiority of LEAP_{KDEy} over LEAP_{ACC} supports the idea of replacing the simple ACC quantification method with a more sophisticated such method as KDEy is beneficial, i.e., shows that improved class prior predictions ultimately lead to improved classifier accuracy prediction.

While S-LEAP and LEAP_{KDEy} perform comparably, S-LEAP tends to achieve slightly better results on average. This is particularly noteworthy because, as shown in Section 4.2, S-LEAP reduces the computational cost to that needed for estimating the class priors (typically equivalent to solving an additional system of n equations), while LEAP_{KDEy} (like LEAP_{ACC}) requires class prior estimation (thus incurring the same cost as S-LEAP) *and* solving a system of n^2 equations. In other words, S-LEAP is $\mathcal{O}(n)$, whereas other LEAP methods are $\mathcal{O}(n^2)$. In the binary case, both systems are computationally efficient at inference time, and the differences in execution times are almost negligible. However, in the multiclass scenario, S-LEAP holds a clear advantage over the other LEAP methods, which do not scale well when the number of classes

⁵For ATC, Mandoline, and RCA, we use the code provided by the authors, while for DoC we use our own implementation since no user-provided code is available. The likely reason why the performance of Mandoline is not competitive, in our experiments, with other baselines is that Mandoline heavily relies on user-defined transformations, called *slicing functions*, which require manual intervention, for each dataset, on the part of the designer. Regarding RCA, it is likely that the “back-and-forth” approach is not able to retain the correct amount of information about the accuracy of h during the training of a clone h' on the pseudo-labelled set.

⁶<https://github.com/HLT-ISTI/QuaPy>

⁷<https://github.com/lorenzovolpi/LEAP>

	Baselines				Our Methods			
	Naïve	ATC	DoC		LEAP _{ACC}	LEAP _{KDEy}	S-LEAP _{KDEy}	O-LEAP _{KDEy}
pageblocks.5	.341 ± .208	.129 ± .090	.084 ± .062	.080 ± .065	.042 ± .038	.035 ± .029	.033 ± .028	
wine-q-white	.120 ± .083	.096 ± .072	.052 ± .038	.058 ± .051	.044 [†] ± .036	.043 ± .035	.044 ± .035	
spambase	.024 ± .018	.025 ± .019	.023 [†] ± .019	.022 [†] ± .017	.031 ± .023	.022 [†] ± .017	.022 ± .017	
ctg.1	.044 ± .032	.032 ± .025	.029 ± .021	.049 ± .035	.032 ± .021	.048 ± .032	.048 ± .032	
ctg.2	.100 ± .076	.076 ± .058	.079 [†] ± .055	.123 ± .075	.132 ± .085	.079 [†] ± .059	.081 ± .055	
ctg.3	.109 ± .076	.034 ± .029	.043 ± .033	.033 ± .027	.042 ± .036	.028 ± .023	.028 [†] ± .024	
wine-q-red	.038 [†] ± .027	.069 ± .049	.050 ± .035	.036 ± .027	.042 ± .033	.039 [†] ± .028	.038 [†] ± .027	
semeion	.127 ± .088	.095 ± .064	.025 ± .022	.057 ± .047	.043 ± .031	.051 ± .040	.034 ± .026	
yeast	.134 ± .093	.092 ± .069	.062 [†] ± .047	.074 ± .061	.063 ± .053	.061 ± .045	.055 ± .044	
cmc.1	.074 ± .048	.082 ± .057	.086 ± .057	.074 ± .055	.074 ± .058	.055 ± .042	.056 ± .042	
cmc.2	.268 ± .196	.177 ± .133	.129 [†] ± .080	.168 ± .128	.131 ± .097	.121 ± .084	.119 ± .082	
cmc.3	.184 ± .126	.165 ± .122	.104 ± .077	.172 ± .139	.099 ± .080	.075 ± .059	.079 ± .067	
german	.127 ± .088	.103 ± .073	.063 [†] ± .044	.088 ± .072	.062 [†] ± .052	.059 [†] ± .046	.058 ± .048	
tictactoe	.017 ± .013	.023 ± .022	.012 ± .010	.013 ± .011	.018 ± .015	.013 ± .011	.013 ± .011	
mammographic	.060 ± .039	.070 ± .042	.087 ± .042	.060 ± .040	.045 ± .033	.060 ± .040	.060 ± .039	
transfusion	.277 ± .201	.203 ± .148	.096 [†] ± .070	.165 ± .140	.091 ± .073	.112 ± .085	.093 [†] ± .075	
breast-cancer	.025 ± .018	.020 ± .016	.014 ± .010	.022 ± .015	.014 [†] ± .011	.022 ± .015	.022 ± .015	
balance.1	.022 ± .014	.033 ± .025	.027 ± .015	.025 ± .018	.032 ± .027	.025 ± .018	.025 ± .017	
balance.3	.028 ± .021	.049 ± .034	.032 ± .026	.019 [†] ± .017	.021 ± .017	.019 ± .016	.019 ± .016	
wdbc	.024 ± .020	.030 ± .022	.030 ± .020	.025 [†] ± .018	.037 ± .031	.026 [†] ± .019	.025 [†] ± .018	
ionosphere	.056 ± .042	.071 ± .047	.099 ± .071	.095 ± .066	.106 ± .069	.084 ± .063	.086 ± .059	
haberman	.223 ± .157	.145 ± .101	.109 ± .086	.189 ± .141	.132 ± .096	.103 [†] ± .085	.098 ± .077	
spectf	.133 ± .097	.096 ± .073	.050 ± .042	.066 ± .058	.057 ± .047	.076 ± .058	.075 ± .052	
sonar	.075 ± .049	.121 ± .080	.165 ± .076	.049 ± .036	.061 ± .046	.069 ± .045	.067 ± .043	
wine.1	.032 ± .000	.028 ± .023	.000 ± .000	.052 ± .029	.123 ± .081	.048 ± .024	.048 ± .026	
wine.2	.019 ± .011	.039 ± .025	.020 [†] ± .011	.022 ± .017	.040 ± .032	.021 [†] ± .016	.022 ± .016	
wine.3	.018 ± .015	.026 ± .022	.026 ± .022	.017 ± .016	.017 ± .015	.017 ± .016	.016 ± .015	
iris.2	.108 ± .070	.072 ± .047	.212 ± .152	.353 ± .119	.220 ± .109	.100 ± .078	.105 ± .074	
iris.3	.032 [†] ± .021	.050 ± .031	.067 ± .025	.031 [†] ± .021	.035 ± .029	.032 [†] ± .022	.030 ± .021	
Average	.098 ± .124	.078 ± .082	.065 ± .072	.077 ± .099	.065 ± .072	.053 [†] ± .055	.052 ± .052	

Table 1 Values of AE obtained in our binary experiments for different CAP methods for the LR classifier. **Boldface** indicates the best method for a given dataset. Superscript [†] denotes the methods (if any) whose scores are not statistically significantly different from the best one according to a Wilcoxon signed-rank test at 0.01 confidence level. Cells are colour-coded so as to facilitate readability and allow for quick comparisons across results, with green indicating best and red indicating worst.

increases. In our experiments, the largest number of classes is reached by the `chess` dataset, with $n = 15$, thus requiring the solution of a system of $n^2 = 225$ independent equations (LEAP_{ACC}, LEAP_{KDEy}), or $(n + 1)^2$ equations in the overconstrained case (O-LEAP). Figure 4 (in Appendix C) displays the run times we clock in our experiments for the different CAP methods. Nevertheless, our experiments also show that O-LEAP clearly outperforms S-LEAP, suggesting that, when the computational cost is affordable, solving the full set of $(n + 1)^2$ equations, as O-LEAP does, is beneficial and leads to a better solution.

Figure 1 shows a selection of prototypical diagonal plots for LR; (the full set of plots for all datasets and all classifiers is online).⁸ These plots display the estimated accuracy as a function of the true accuracy, with each dot representing a set of test datapoints; a perfect CAP method would give rise only to values lying on the diagonal from (0,0) to (1,1). For the sake of clarity, we have omitted methods Naïve (which returns dots all lying on a horizontal line, and is thus uninteresting), ATC, and LEAP_{ACC}. Note that O-LEAP tends to display the smallest deviation from the main diagonal.

⁸<https://github.com/lorenzovolpi/LEAP/tree/main/output/plots>

	Baselines				Our Methods			
	Naïve	ATC	DoC		LEAP _{ACC}	LEAP _{KDEy}	S-LEAP _{KDEy}	O-LEAP _{KDEy}
poker-hand	.375 ± .102	.368 ± .114	.082 ± .065		.100 ± .079	.118 ± .104	.107 ± .097	.100 ± .088
connect-4	.254 ± .185	.164 ± .132	.093 ± .070		.242 ± .188	.066 ± .052	.078 ± .061	.084 ± .065
shuttle	.250 ± .188	.230 ± .187	.151 ± .101		.128 ± .104	.038 ± .031	.034 ± .029	.034 [†] ± .029
chess	.052 ± .040	.085 ± .061	.045 ± .033		.056 ± .043	.044 ± .033	.044 [†] ± .032	.041 ± .031
dry-bean	.024 ± .017	.022 ± .018	.019 ± .014		.021 ± .015	.026 ± .019	.020 ± .015	.020 [†] ± .015
nursery	.082 ± .070	.042 ± .037	.033 ± .027		.038 ± .035	.034 ± .025	.029 ± .026	.022 ± .017
hand-digits	.021 ± .016	.019 ± .015	.017 ± .013		.020 ± .015	.032 ± .019	.019 ± .014	.019 ± .013
satellite	.070 ± .062	.037 ± .030	.036 ± .027		.044 ± .036	.052 ± .036	.030 ± .023	.030 [†] ± .023
digits	.016 ± .013	.016 ± .014	.014 [†] ± .012		.015 ± .012	.026 ± .015	.015 ± .012	.014 ± .010
page-block	.260 ± .135	.178 ± .117	.101 ± .087		.141 ± .094	.088 ± .059	.104 ± .068	.074 ± .051
waveform-v1	.031 ± .023	.038 ± .029	.030 ± .023		.028 ± .021	.038 ± .028	.028 ± .021	.028 ± .021
academic-success	.139 ± .120	.078 ± .062	.082 ± .056		.077 ± .065	.055 [†] ± .045	.051 ± .041	.052 [†] ± .040
abalone	.057 ± .042	.068 ± .054	.045 ± .035		.075 ± .053	.058 ± .043	.039 ± .030	.040 [†] ± .031
molecular	.027 ± .022	.025 ± .021	.022 ± .018		.024 ± .019	.026 ± .020	.025 ± .019	.024 ± .018
image-seg	.030 ± .023	.023 ± .019	.020 [†] ± .016		.019 ± .017	.030 ± .023	.018 ± .015	.019 ± .015
obesity	.052 ± .037	.049 ± .039	.033 ± .025		.038 ± .030	.062 ± .041	.034 [†] ± .026	.036 [†] ± .027
cmc	.079 ± .053	.065 ± .048	.047 ± .034		.057 ± .048	.051 ± .040	.050 ± .036	.045 ± .034
hcv	.044 [†] ± .032	.079 ± .054	.042 ± .031		.061 ± .044	.056 ± .041	.048 ± .035	.049 ± .036
phishing	.203 ± .179	.133 ± .119	.095 ± .075		.144 ± .116	.054 ± .041	.046 [†] ± .037	.045 ± .036
yeast	.074 ± .052	.086 ± .062	.080 ± .055		.082 ± .058	.067 ± .052	.057 ± .044	.059 ± .045
mhr	.099 ± .061	.071 ± .052	.083 ± .053		.078 ± .061	.058 ± .046	.058 ± .043	.052 ± .041
Average	.107 ± .134	.089 ± .113	.056 ± .060		.071 ± .088	.051 ± .048	.045 ± .047	.042 ± .044

Table 2 Values of AE obtained in our multiclass experiments for different CAP methods over a logistic regression classifier. **Boldface**, superscript †, and colour coding are used in the same way as in Table 1.

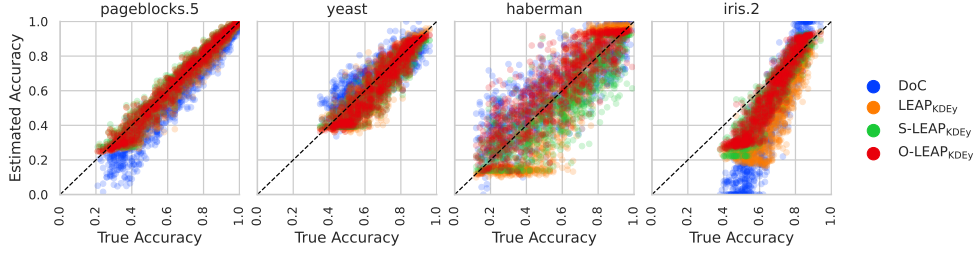


Fig. 1 Diagonal plots illustrating the correlation between true accuracy (x-axis) and estimated accuracy (y-axis) of a classifier trained via LR, for different CAP methods.

To better understand how performance depends on the quantifier, we compare O-LEAP_{KDEy} with O-LEAP_{CC} (i.e., the version of O-LEAP that, as the quantification method, employs the simplistic “classify and count”) and O-LEAP_Φ (the version of O-LEAP endowed with an oracle that always predicts the true prevalence values). As shown in Figure 2, O-LEAP_{CC} performs markedly worse than O-LEAP, thus confirming that KDEy plays a key role in the effectiveness of O-LEAP_{KDEy}. While O-LEAP_Φ is an ideal upper bound for O-LEAP, the results show that the gap between O-LEAP_Φ and O-LEAP_{KDEy} is relatively narrow, which indicates that O-LEAP_{KDEy} is a good method. Aside from this, given that the experiments are designed to generate PPS, any deviation of O-LEAP_Φ from the main diagonal must be caused by random error

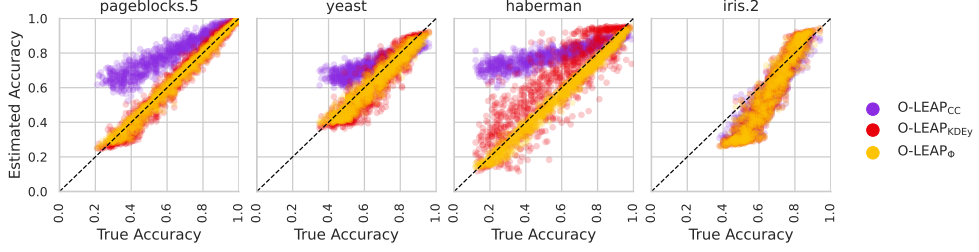


Fig. 2 Diagonal plots showing the same experimental setup as in Figure 1, but comparing O-LEAP_{KDEy} with O-LEAP_{CC} (lower-bound) and O-LEAP_Φ (upper-bound).

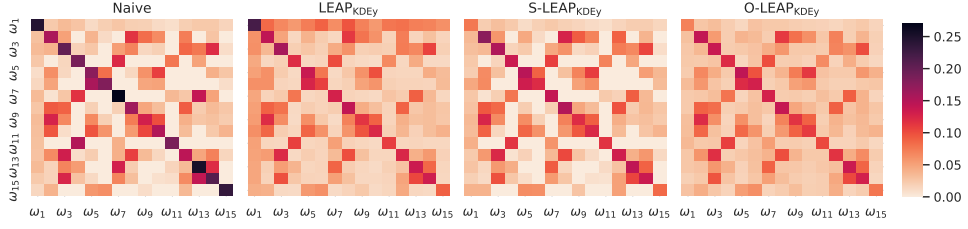


Fig. 3 Heat maps representing the prediction errors $e_{ij} = |\hat{c}_{ij}^U - c_{ij}^U|$ for the contingency table of dataset chess (with $n = 15$ classes) for classifier MLP. The errors e_{ij} have been transformed using $\sqrt{e_{ij}}$ so as to enhance contrast and improve visualization.

sampling; indeed, O-LEAP_Φ shows the greatest deviation from the diagonal in the smallest dataset, i.e., iris.2.

Figure 3 displays the prediction errors $e_{ij} = |\hat{c}_{ij}^U - c_{ij}^U|$ for the entries of the test contingency table $\mathbf{U}$ for dataset chess and for different methods. Naïve corresponds to the error generated by assuming that the training set and the test set are IID, i.e., by disregarding the presence of PPS. This method tends to make most of the errors along the diagonal ($e_{11}, \dots, e_{nn}$), which is critical for computing Acc (as it corresponds to the numerator of the measure), thus resulting in large CAP error. Conversely, LEAP_{KDEy} tends to allocate more errors across the first row ($e_{11}, \dots, e_{1n}$) and the first column ($e_{11}, \dots, e_{n1}$); this is explained by the fact that this method drops the constraints associated with class ω_1 from all equation groups (Section 4.2), thus misrepresenting this class relative to the others. S-LEAP does not suffer from this issue but exhibits higher variance in errors. Finally, O-LEAP seems to produce a more homogeneous distribution of errors and, in particular, tends to yield fewer errors along the diagonal when compared to the other methods.

6 Conclusions

In this paper we have proposed LEAP, a family of methods for classifier accuracy prediction (CAP) specifically designed to address situations affected by prior probability shift (PPS). Our methods work by estimating the entries of the contingency table that results from classifying the test set, from which any evaluation measure can be computed. As a result, and in contrast to previously existing method, LEAP is not limited to any specific evaluation measure, and can compute more than one such measure at the same time, without requiring retraining. We have also drawn a connection with the field of quantification, and through several experiments we have shown that, when our CAP methods are endowed with state-of-the-art quantification methods, they tends to outperform existing CAP methods.

We have also shown that the problem is overconstrained, and have discussed different strategies for selecting the system of equations that, once solved, provides the required estimates. In particular, O-LEAP, which considers *all* the constraints governing the problem, tends to achieve the best results. Additionally, we have shown that some of our proposed methods do not scale well when the number of classes is large; in such cases, our S-LEAP method should be preferred to O-LEAP, since it strikes a good balance between CAP accuracy and computational cost.

Possible directions for future work include considering multi-objective optimization problems, in which our method could be especially useful due to its ability to estimate more than one evaluation measure simultaneously. Additionally, we aim to extend our methods to other types of dataset shift, incorporating different assumptions about data distributions, such as covariate shift. Finally, we plan to explore their effectiveness in estimating the accuracy of classifiers trained via deep learning.

References

- [1] Storkey, A.: When training and test sets are different: Characterizing learning transfer. In: Quiñero-Candela, J., Sugiyama, M., Schwaighofer, A., Lawrence, N.D. (eds.) *Dataset Shift in Machine Learning*, pp. 3–28. The MIT Press, Cambridge, US (2009)
- [2] Forman, G.: Counting positives accurately despite inaccurate classification. In: *Proceedings of the 16th European Conference on Machine Learning (ECML 2005)*, Porto, PT, pp. 564–575 (2005). https://doi.org/10.1007/11564096_55
- [3] González, P., Castaño, A., Chawla, N.V., Coz, J.J.: A review on quantification learning. *ACM Computing Surveys* **50**(5), 74–17440 (2017) <https://doi.org/10.1145/3117807>
- [4] Esuli, A., Fabris, A., Moreo, A., Sebastiani, F.: *Learning to Quantify*. Springer, Cham, CH (2023). <https://doi.org/10.1007/978-3-031-20467-8>
- [5] Volpi, L., Moreo, A., Sebastiani, F.: A simple method for classifier accuracy prediction under prior probability shift. In: *Proceedings of the 27th International*

- Conference on Discovery Science (DS 2024), Pisa, IT, pp. 267–283 (2024). https://doi.org/10.1007/978-3-031-78980-9_17
- [6] Garg, S., Balakrishnan, S., Lipton, Z.C., Neyshabur, B., Sedghi, H.: Leveraging unlabeled data to predict out-of-distribution performance. In: Proceedings of the 10th International Conference on Learning Representations (ICLR 2022), Virtual Event (2022)
 - [7] Guillory, D., Shankar, V., Ebrahimi, S., Darrell, T., Schmidt, L.: Predicting with confidence on unseen distributions. In: Proceedings of the IEEE/CVF International Conference on Computer Vision (ICCV 2021), Montreal, CA, pp. 1134–1144 (2021)
 - [8] Elshahar, H., Gallé, M.: To annotate or not? Predicting performance drop under domain shift. In: Proceedings of the 2019 Conference on Empirical Methods in Natural Language Processing and the 9th International Joint Conference on Natural Language Processing (EMNLP-IJCNLP 2019), Hong Kong, CN, pp. 2163–2173 (2019). <https://doi.org/10.18653/v1/D19-1222>
 - [9] Chen, M.F., Goel, K., Sohoni, N.S., Poms, F., Fatahalian, K., Ré, C.: Mandoline: Model evaluation under distribution shift. In: Proceedings of the 38th International Conference on Machine Learning (ICML 2021), Virtual Event, pp. 1617–1629 (2021)
 - [10] Chen, J., Liu, F., Avci, B., Wu, X., Liang, Y., Jha, S.: Detecting errors and estimating accuracy on unlabeled data with self-training ensembles. In: Proceedings of the 35th Conference on Neural Information Processing Systems (NeurIPS 2021), Virtual Event, pp. 14980–14992 (2021)
 - [11] Jiang, Y., Nagarajan, V., Baek, C., Kolter, J.Z.: Assessing generalization of SGD via disagreement. In: Proceedings of the International Conference on Learning Representations (ICLR 2022), Virtual Event (2022)
 - [12] Sugiyama, M., Nakajima, S., Kashima, H., Buenau, P., Kawanabe, M.: Direct importance estimation with model selection and its application to covariate shift adaptation. In: Proceedings of the 21st Conference on Advances in Neural Information Processing Systems (NIPS 2007), Vancouver, CA, pp. 1433–1440 (2007)
 - [13] Zhang, K., Schölkopf, B., Muandet, K., Wang, Z.: Domain adaptation under target and conditional shift. In: Proceedings of the 30th International Conference on Machine Learning (ICML 2013), Atlanta, US, pp. 819–827 (2013)
 - [14] Lipton, Z.C., Wang, Y., Smola, A.J.: Detecting and correcting for label shift with black box predictors. In: Proceedings of the 35th International Conference on Machine Learning (ICML 2018), Stockholm, SE, pp. 3128–3136 (2018)

- [15] Schölkopf, B., Janzing, D., Peters, J., Sgouritsa, E., Zhang, K., Mooij, J.M.: On causal and anticausal learning. In: Proceedings of the 29th International Conference on Machine Learning (ICML 2012), Edinburgh, UK (2012)
- [16] Tasche, D.: Comments on Friedman’s method for class distribution estimation. arXiv:2405.16666 [cs.LG] (2024)
- [17] Forman, G.: Quantifying counts and costs via classification. *Data Mining and Knowledge Discovery* **17**(2), 164–206 (2008) <https://doi.org/10.1007/s10618-008-0097-y>
- [18] Bella, A., Ferri, C., Hernández-Orallo, J., Ramírez-Quintana, M.J.: Quantification via probability estimators. In: Proceedings of the 11th IEEE International Conference on Data Mining (ICDM 2010), Sydney, AU, pp. 737–742 (2010). <https://doi.org/10.1109/icdm.2010.75>
- [19] Bunse, M.: On multi-class extensions of adjusted classify and count. In: Proceedings of the 2nd International Workshop on Learning to Quantify (LQ 2022), Grenoble, IT, pp. 43–50 (2022)
- [20] Moreo, A., González, P., del Coz, J.J.: Kernel density estimation for multi-class quantification. *Machine Learning* **114**(4) (2025) <https://doi.org/10.1007/s10994-024-06726-5>
- [21] Smith, N.A., Tromble, R.W.: Sampling uniformly from the unit simplex. Technical report, Johns Hopkins University (2004). <https://www.cs.cmu.edu/~nasmith/papers/smith+tromble.tr04.pdf>
- [22] Kelly, M., Longjohn, R., Nottingham, K.: The UCI machine learning repository. <https://archive.ics.uci.edu>
- [23] Pedregosa, F., Varoquaux, G., Gramfort, A., Michel, V., Thirion, B., Grisel, O., Blondel, M., Prettenhofer, P., Weiss, R., Dubourg, V., Vanderplas, J., Passos, A., Cournapeau, D., Brucher, M., Perrot, M., Duchesnay, E.: Scikit-learn: Machine learning in Python. *Journal of Machine Learning Research* **12**, 2825–2830 (2011)
- [24] Moreo, A., Esuli, A., Sebastiani, F.: QuaPy: A Python-based framework for quantification. In: Proceedings of the 30th ACM International Conference on Knowledge Management (CIKM 2021), Gold Coast, AU, pp. 4534–4543 (2021). <https://doi.org/10.1145/3459637.3482015>

Declarations

Acknowledgments and Funding. Lorenzo Volpi’s work was supported by project “Italian Strengthening of ESFRI RI RESILIENCE” (ITSERR), funded by the European Union under the NextGenerationEU funding scheme (CUP B53C22001770006). Alejandro Moreo’s and Fabrizio Sebastiani’s work was partially supported by project “Future Artificial Intelligence Research” (FAIR), project “Quantification under Dataset Shift” (QuaDaSh), and project “Strengthening the Italian RI for Social Mining and Big Data Analytics” (SoBigData.it), all funded by the European Union under the NextGenerationEU funding scheme (CUP B53D22000980006, CUP B53D23026250001, CUP B53C22001760006, respectively).

Conflicts of interest/Competing interests. The authors have no relevant financial or non-financial interests to disclose.

Ethics approval and consent to participate. Not applicable.

Consent for publication. Not applicable.

Data availability. All datasets are publicly available from the referred sources.

Code availability. The code to reproduce all the experiments is available at <https://github.com/lorenzovolpi/LEAP>.

Author contributions. All authors contributed to the main idea. L.V. conducted the experiments. A.M. and F.S. wrote the first draft of the manuscript. All authors contributed to the final writing and reviewed the manuscript.

A The Closed Form of F_1

In Section 4.3, we have shown the closed-form solution of LEAP_{ACC} for the case in which we choose vanilla accuracy (Acc) as our classifier accuracy measure. An analogous closed-form solution of LEAP_{ACC} can be given for the case in which classifier accuracy must be measured via the well-known F_1 measure (Equation 27). Following the same rationale, F_1 can be expressed in compact form using γ and $\mathbf{q}$ (Equation 28), and expanded as a function that uses only γ along with counts from the validation contingency table $\mathbf{V}$ (Equation 29), i.e.,

$$F_1(h, U) = \frac{2 \cdot \text{TP}^U}{2 \cdot \text{TP}^U + \text{FP}^U + \text{FN}^U} \quad (27)$$

$$= \frac{2q_{\oplus} \cdot \text{tpr}^V}{\gamma_{\oplus} + q_{\oplus}} \quad (28)$$

$$= \frac{2 \cdot \text{TP}^V (\gamma_{\oplus} \cdot \text{TN}^V - \gamma_{\ominus} \cdot \text{FP}^V)}{\gamma_{\oplus} \cdot \text{TN}^V (2 \cdot \text{TP}^V + \text{FN}^V) - \text{FP}^V (\text{FN}^V + \gamma_{\ominus} \cdot \text{TP}^V)} \quad (29)$$

B Results for Classifiers Trained via k NN, SVMs, MLP

In this appendix we report the results of the complete set of experiments we have carried out for the classifiers trained via k NN (Tables 3 and 4), SVMs (Tables 5 and 6), and MLP (Tables 7 and 8).

	Baselines			Our Methods			
	Naïve	ATC	DoC	LEAP _{ACC}	LEAP _{KDEy}	S-LEAP _{KDEy}	O-LEAP _{KDEy}
pageblocks.5	.355 ± .217	.242 ± .148	.041 ± .034	.094 ± .077	.049 ± .045	.037 ± .033	.038 ± .033
wine-q-white	.077 ± .054	.047 ± .037	.042 ± .032	.041 ± .035	.047 ± .037	.038 [†] ± .030	.037 ± .030
spambase	.045 ± .031	.074 ± .048	.029 [†] ± .023	.029 ± .021	.038 ± .031	.027 [†] ± .020	.027 ± .020
ctg.1	.068 ± .052	.100 ± .062	.060 ± .041	.049 ± .039	.024 ± .018	.043 ± .033	.043 ± .033
ctg.2	.170 ± .125	.184 ± .107	.029 ± .022	.116 ± .083	.100 ± .078	.062 ± .052	.058 ± .048
ctg.3	.203 ± .136	.126 ± .085	.071 ± .049	.070 ± .060	.044 ± .036	.053 ± .045	.040 ± .035
wine-q-red	.039 [†] ± .030	.147 ± .061	.053 ± .037	.036 ± .028	.044 ± .035	.040 ± .030	.038 [†] ± .029
semeion	.149 ± .105	.085 ± .059	.031 ± .025	.041 ± .037	.031 [†] ± .029	.032 ± .028	.027 ± .022
yeast	.137 ± .097	.109 ± .078	.072 [†] ± .053	.143 ± .097	.107 ± .078	.072 ± .055	.066 ± .050
cmc.1	.149 ± .091	.230 ± .147	.131 ± .094	.094 ± .079	.062 ± .049	.064 [†] ± .046	.068 ± .047
cmc.2	.249 ± .181	.346 ± .208	.089 ± .068	.172 ± .142	.102 ± .074	.108 ± .080	.104 ± .077
cmc.3	.177 ± .120	.247 ± .164	.125 ± .079	.129 ± .105	.083 ± .068	.094 ± .071	.093 ± .072
german	.146 ± .100	.237 ± .124	.094 ± .068	.138 ± .096	.082 ± .064	.075 [†] ± .058	.073 ± .055
tictactoe	.075 ± .052	.069 ± .046	.052 ± .033	.030 ± .026	.028 ± .030	.024 ± .020	.021 ± .019
mammographic	.055 [†] ± .037	.104 ± .047	.056 [†] ± .037	.055 ± .037	.066 ± .044	.055 ± .037	.055 ± .037
transfusion	.229 ± .166	.190 ± .132	.074 ± .058	.163 ± .120	.096 ± .077	.080 [†] ± .060	.087 ± .068
breast-cancer	.017 ± .011	.024 ± .018	.020 ± .012	.017 ± .011	.012 ± .010	.017 ± .010	.017 ± .010
balance.1	.039 ± .025	.049 ± .035	.068 ± .036	.031 ± .025	.040 ± .037	.029 ± .023	.030 ± .023
balance.3	.026 ± .021	.071 ± .031	.030 ± .024	.021 [†] ± .017	.026 ± .022	.021 ± .017	.021 ± .017
wdbc	.044 ± .032	.033 ± .024	.029 ± .021	.031 [†] ± .022	.029 [†] ± .023	.032 [†] ± .023	.031 [†] ± .022
ionosphere	.153 ± .103	.056 ± .048	.083 ± .050	.071 ± .056	.065 ± .051	.073 ± .052	.073 ± .052
haberman	.236 ± .167	.173 ± .128	.174 ± .111	.266 ± .201	.106 ± .082	.090 ± .072	.107 ± .085
spectf	.216 ± .157	.257 ± .130	.132 ± .077	.379 ± .203	.082 ± .066	.096 ± .069	.296 ± .175
sonar	.140 ± .092	.118 ± .085	.120 ± .085	.190 ± .118	.174 ± .108	.103 ± .054	.092 ± .050
wine.1	.012 ± .008	.014 [†] ± .014	.014 [†] ± .014	.023 ± .015	.087 ± .061	.021 ± .014	.022 ± .015
wine.2	.084 ± .032	.090 [†] ± .060	.137 ± .056	.150 ± .078	.198 ± .108	.128 ± .072	.123 ± .069
wine.3	.034 ± .026	.053 ± .037	.047 ± .032	.033 ± .028	.014 ± .014	.033 ± .027	.026 ± .021
iris.2	.059 ± .015	.017 ± .016	.044 ± .030	.086 ± .046	.127 ± .082	.083 ± .049	.082 ± .046
iris.3	.121 ± .091	.037 ± .030	.037 [†] ± .028	.085 ± .063	.042 ± .031	.056 ± .046	.044 ± .032
Average	.121 ± .130	.122 ± .126	.068 ± .066	.096 ± .117	.069 ± .073	.058 ± .055	.063 ± .076

Table 3 CAP results, in terms of AE, for binary classifiers trained via k NN.

	Baselines			Our Methods			
	Naïve	ATC	DoC	LEAP _{ACC}	LEAP _{KDEy}	S-LEAP _{KDEy}	O-LEAP _{KDEy}
poker-hand	.391 ± .084	.424 ± .091	.066 [†] ± .050	.093 ± .072	.092 ± .068	.101 ± .062	.065 ± .052
connect-4	.238 ± .161	.183 ± .125	.083 ± .061	.118 ± .096	.075 ± .060	.063 [†] ± .050	.060 ± .045
shuttle	.185 ± .146	.085 ± .072	.072 ± .065	.062 ± .057	.033 ± .029	.036 ± .033	.031 ± .028
chess	.076 ± .049	.069 ± .050	.044 ± .033	.054 ± .040	.067 ± .046	.039 [†] ± .029	.038 ± .027
dry-bean	.025 ± .018	.040 ± .026	.019 ± .014	.021 ± .016	.029 ± .020	.021 ± .015	.021 ± .015
nursery	.161 ± .140	.184 ± .135	.081 ± .061	.080 ± .065	.055 ± .039	.032 ± .024	.034 ± .025
hand-digits	.011 ± .007	.011 ± .011	.009 ± .007	.010 ± .008	.016 ± .008	.010 ± .008	.009 [†] ± .006
satellite	.041 ± .032	.066 ± .040	.028 ± .021	.036 ± .030	.058 ± .040	.030 [†] ± .023	.031 [†] ± .024
digits	.018 ± .014	.017 [†] ± .015	.017 ± .014	.017 ± .014	.030 ± .017	.017 [†] ± .014	.015 ± .012
page-block	.325 ± .171	.239 ± .165	.122 ± .099	.153 ± .126	.070 ± .052	.090 ± .069	.064 ± .047
waveform-v1	.031 [†] ± .023	.078 ± .040	.029 ± .022	.030 [†] ± .023	.036 ± .026	.030 [†] ± .023	.029 [†] ± .022
academic-success	.174 ± .136	.169 ± .117	.081 ± .059	.156 ± .135	.055 [†] ± .043	.056 [†] ± .043	.052 ± .040
abalone	.059 ± .041	.074 ± .051	.046 ± .036	.055 ± .042	.056 ± .042	.042 ± .031	.040 ± .030
molecular	.048 ± .034	.095 ± .050	.042 ± .030	.032 ± .024	.040 ± .032	.031 [†] ± .023	.030 ± .022
image-seg	.045 ± .037	.036 ± .032	.032 ± .025	.029 ± .024	.037 ± .026	.025 ± .022	.022 ± .018
obesity	.069 ± .051	.047 ± .037	.039 ± .029	.050 ± .042	.058 ± .043	.035 [†] ± .027	.034 ± .027
cmc	.052 ± .039	.295 ± .068	.057 ± .042	.068 ± .046	.083 ± .057	.050 ± .038	.046 ± .036
hcv	.052 ± .035	.569 ± .063	.045 ± .033	.064 ± .043	.053 ± .041	.055 ± .037	.052 ± .036
phishing	.204 ± .178	.115 ± .097	.064 ± .049	.111 ± .090	.043 ± .033	.039 ± .029	.034 ± .025
yeast	.069 ± .049	.089 ± .064	.065 ± .045	.055 ± .045	.056 ± .043	.057 ± .043	.049 ± .037
mhr	.067 ± .045	.076 ± .053	.068 ± .045	.051 ± .042	.053 ± .046	.050 ± .038	.046 ± .036
Average	.112 ± .137	.141 ± .158	.053 ± .052	.064 ± .074	.052 ± .045	.043 ± .042	.038 ± .035

Table 4 CAP results, in terms of AE, for multiclass classifiers trained via k NN.

	Baselines			Our Methods			
	Naïve	ATC	DoC	LEAP _{ACC}	LEAP _{KDEy}	S-LEAP _{KDEy}	O-LEAP _{KDEy}
pageblocks.5	.372 ± .227	.200 ± .123	.184 ± .122	.073 ± .060	.089 ± .065	.106 ± .064	.103 ± .063
wine-q-white	.083 ± .058	.049 ± .040	.042 ± .033	.042 ± .036	.044 ± .034	.037 [†] ± .030	.036 ± .029
spambase	.028 ± .020	.028 ± .022	.028 ± .022	.022 ± .017	.031 ± .024	.023 [†] ± .016	.022 [†] ± .016
ctg.1	.071 ± .053	.049 ± .039	.031 ± .025	.037 ± .028	.025 ± .019	.027 [†] ± .020	.026 [†] ± .020
ctg.2	.167 ± .124	.037 ± .029	.069 ± .053	.109 ± .078	.065 ± .055	.040 [†] ± .034	.038 [†] ± .033
ctg.3	.194 ± .127	.102 ± .066	.035 ± .029	.081 ± .059	.090 ± .069	.106 ± .071	.106 ± .071
wine-q-red	.046 ± .032	.060 ± .043	.038 ± .028	.041 ± .031	.042 [†] ± .032	.047 ± .033	.045 ± .033
semeion	.149 ± .103	.082 ± .059	.022 ± .018	.084 ± .065	.067 ± .057	.079 ± .059	.079 ± .058
yeast	.143 ± .098	.076 ± .052	.072 ± .052	.079 ± .065	.071 ± .058	.059 ± .044	.052 ± .041
cmc.1	.094 ± .061	.097 ± .070	.079 ± .057	.052 ± .042	.048 [†] ± .039	.049 ± .036	.047 ± .035
cmc.2	.290 ± .212	.210 ± .155	.222 ± .135	.233 ± .180	.121 ± .087	.150 ± .105	.131 ± .087
cmc.3	.167 ± .109	.091 ± .066	.168 ± .118	.164 ± .115	.124 ± .081	.122 ± .083	.117 ± .076
german	.203 ± .138	.154 ± .108	.094 ± .065	.150 ± .117	.086 ± .065	.111 ± .084	.109 ± .082
tictactoe	.030 ± .025	.019 ± .017	.026 ± .021	.015 ± .014	.021 ± .019	.025 ± .022	.022 ± .021
mammographic	.063 ± .044	.078 ± .045	.069 ± .047	.055 ± .040	.078 ± .045	.065 ± .040	.065 ± .040
transfusion	.269 ± .194	.206 ± .151	.098 ± .076	.245 ± .179	.103 [†] ± .076	.130 ± .092	.102 [†] ± .078
breast-cancer	.026 ± .012	.014 ± .012	.030 ± .015	.031 ± .013	.020 ± .015	.028 ± .013	.028 ± .013
balance.1	.022 [†] ± .018	.027 ± .020	.022 [†] ± .018	.020 ± .016	.027 ± .023	.023 ± .017	.022 ± .016
balance.3	.055 ± .019	.033 ± .023	.058 ± .024	.045 ± .021	.038 ± .025	.037 ± .017	.037 ± .017
wdbc	.024 ± .019	.026 ± .021	.043 ± .030	.024 ± .018	.031 ± .021	.021 ± .017	.020 ± .016
ionosphere	.025 ± .016	.031 ± .023	.048 ± .035	.034 ± .026	.038 ± .028	.030 ± .021	.046 ± .032
haberman	.239 ± .169	.122 ± .081	.087 ± .070	.185 ± .149	.175 ± .121	.118 ± .091	.129 ± .099
spectf	.310 ± .224	.178 ± .130	.069 ± .051	.180 ± .146	.151 ± .113	.268 ± .159	.179 ± .131
sonar	.131 ± .066	.041 ± .030	.176 ± .085	.113 ± .043	.168 ± .089	.116 ± .041	.117 ± .039
wine.1	.016 ± .000	.000 ± .000	.000 ± .000	.025 ± .014	.087 ± .062	.023 ± .013	.027 ± .011
wine.2	.024 ± .017	.012 ± .012	.038 ± .019	.021 ± .016	.016 ± .013	.021 ± .016	.019 ± .014
wine.3	.018 ± .015	.026 ± .022	.016 ± .016	.017 ± .016	.020 ± .017	.026 ± .022	.022 ± .018
iris.2	.044 ± .022	.034 ± .026	.022 ± .014	.048 ± .032	.092 ± .057	.055 ± .033	.054 ± .031
iris.3	.066 ± .051	.039 ± .026	.116 ± .059	.036 ± .027	.032 [†] ± .027	.030 ± .024	.031 ± .025
Average	.116 ± .145	.073 ± .091	.069 ± .079	.078 ± .101	.069 ± .073	.068 ± .079	.063 ± .068

Table 5 CAP results, in terms of AE, for binary classifiers trained via SVMs.

	Baselines			Our Methods			
	Naïve	ATC	DoC	LEAP _{ACC}	LEAP _{KDEy}	S-LEAP _{KDEy}	O-LEAP _{KDEy}
poker-hand	.404 ± .090	.424 ± .109	.078 ± .052	.069 ± .054	.081 ± .064	.095 ± .072	.086 ± .065
connect-4	.263 ± .189	.159 ± .124	.081 ± .059	.186 ± .152	.098 ± .075	.070 ± .055	.084 ± .066
shuttle	.208 ± .162	.137 ± .107	.069 ± .062	.063 [†] ± .055	.057 ± .053	.058 ± .049	.061 ± .053
chess	.094 ± .058	.075 ± .055	.060 ± .043	.054 [†] ± .042	.051 [†] ± .039	.063 ± .043	.051 ± .038
dry-bean	.023 ± .017	.021 ± .016	.018 ± .013	.021 ± .016	.024 ± .017	.020 ± .015	.020 ± .014
nursery	.124 ± .105	.115 ± .096	.116 ± .097	.060 ± .060	.095 ± .086	.114 ± .096	.108 ± .092
hand-digits	.008 ± .006	.006 ± .007	.006 [†] ± .005	.008 ± .006	.010 ± .007	.008 ± .006	.007 ± .005
satellite	.051 ± .042	.033 ± .027	.027 ± .021	.034 ± .027	.041 ± .030	.028 [†] ± .021	.028 [†] ± .021
digits	.015 ± .013	.014 ± .012	.014 ± .011	.014 ± .011	.019 ± .012	.015 ± .013	.011 ± .008
page-block	.303 ± .167	.159 ± .109	.122 ± .092	.152 ± .111	.072 ± .051	.100 ± .065	.084 ± .055
waveform-v1	.031 ± .024	.037 ± .029	.033 ± .024	.031 ± .024	.039 ± .030	.029 [†] ± .022	.029 ± .022
academic-success	.152 ± .120	.090 ± .068	.063 ± .044	.087 ± .072	.057 ± .045	.058 ± .046	.052 ± .039
abalone	.060 ± .045	.084 ± .061	.047 ± .037	.083 ± .059	.055 ± .041	.052 ± .039	.047 [†] ± .036
molecular	.022 ± .016	.030 ± .022	.020 ± .015	.023 ± .017	.025 ± .018	.023 ± .016	.023 ± .016
image-seg	.038 ± .034	.039 ± .038	.032 ± .028	.025 [†] ± .021	.023 ± .018	.027 ± .024	.025 [†] ± .021
obesity	.046 ± .034	.041 ± .031	.037 ± .028	.042 ± .032	.064 ± .040	.031 ± .024	.034 ± .025
cmc	.064 ± .047	.059 [†] ± .046	.062 ± .045	.067 ± .052	.067 ± .051	.052 ± .040	.055 ± .042
hcv	.044 ± .033	.055 ± .042	.054 ± .041	.139 ± .090	.114 ± .081	.125 ± .087	.135 ± .091
phishing	.218 ± .195	.118 ± .104	.072 ± .052	.155 ± .130	.045 [†] ± .035	.048 ± .037	.043 ± .035
yeast	.068 ± .047	.071 ± .054	.078 ± .056	.058 ± .043	.062 ± .047	.056 ± .042	.055 ± .042
mhr	.105 ± .063	.078 ± .055	.099 ± .059	.056 ± .046	.061 [†] ± .052	.057 [†] ± .042	.061 ± .046
Average	.111 ± .141	.088 ± .111	.056 ± .058	.068 ± .082	.055 ± .055	.054 ± .057	.052 ± .056

Table 6 CAP results, in terms of AE, for multiclass classifiers trained via SVMs.

	Baselines				Our Methods			
	Naïve	ATC	DoC	LEAP _{ACC}	LEAP _{KDEy}	S-LEAP _{KDEy}	O-LEAP _{KDEy}	
pageblocks.5	.221 ± .137	.144 ± .097	.070 ± .060	.062 ± .056	.038 ± .031	.041 ± .036	.039 ± .032	
wine-q-white	.072 ± .053	.058 ± .045	.046 ± .035	.041 ± .035	.044 ± .035	.038 ± .030	.037 ± .030	
spambase	.028 ± .022	.026 ± .020	.025 ± .019	.024 ± .019	.027 ± .021	.023 [†] ± .019	.023 ± .018	
ctg.1	.057 ± .042	.029 ± .024	.031 ± .026	.025 ± .021	.024 ± .018	.023 [†] ± .019	.022 ± .018	
ctg.2	.146 ± .106	.091 ± .067	.048 ± .041	.049 ± .041	.051 ± .045	.035 ± .028	.036 ± .030	
ctg.3	.099 ± .067	.067 ± .049	.086 ± .056	.049 ± .039	.038 ± .030	.046 ± .036	.040 ± .032	
wine-q-red	.056 ± .039	.054 ± .041	.044 ± .031	.046 [†] ± .033	.054 ± .042	.050 ± .035	.048 ± .033	
semeion	.057 ± .046	.028 ± .025	.041 ± .031	.024 ± .020	.044 ± .036	.021 [†] ± .018	.021 ± .018	
yeast	.091 ± .064	.068 ± .050	.059 ± .046	.050 ± .042	.049 ± .042	.045 ± .035	.043 ± .034	
cmc.1	.062 ± .045	.066 ± .050	.062 ± .046	.049 ± .039	.062 ± .046	.045 ± .035	.042 ± .033	
cmc.2	.160 ± .117	.081 ± .064	.058 ± .043	.124 ± .084	.092 ± .071	.102 ± .065	.104 ± .069	
cmc.3	.088 ± .063	.059 [†] ± .047	.071 ± .050	.072 ± .053	.059 [†] ± .049	.065 ± .047	.057 ± .042	
german	.103 ± .068	.079 ± .054	.088 ± .056	.067 ± .058	.072 ± .057	.054 ± .039	.048 ± .036	
tictactoe	.015 ± .011	.014 [†] ± .012	.013 ± .009	.012 ± .010	.014 ± .011	.012 ± .010	.013 ± .010	
mammographic	.063 ± .044	.092 ± .044	.065 ± .043	.063 ± .044	.041 ± .030	.063 ± .044	.062 ± .044	
transfusion	.163 ± .121	.114 ± .089	.060 ± .047	.080 ± .070	.066 [†] ± .059	.084 ± .068	.073 ± .061	
breast-cancer	.026 ± .012	.018 ± .016	.021 ± .014	.034 ± .019	.011 ± .010	.034 ± .018	.034 ± .018	
balance.1	.026 ± .019	.029 ± .020	.032 ± .021	.026 ± .019	.020 ± .017	.026 ± .019	.022 [†] ± .016	
balance.3	.012 ± .007	.013 [†] ± .012	.012 [†] ± .013	.011 ± .007	.025 ± .017	.011 [†] ± .007	.011 [†] ± .007	
wdbc	.058 ± .028	.063 ± .029	.044 ± .026	.050 ± .029	.028 ± .024	.052 ± .030	.049 ± .028	
ionosphere	.049 ± .030	.040 ± .028	.126 ± .071	.084 ± .053	.092 ± .059	.084 ± .056	.084 ± .053	
haberman	.085 ± .058	.154 ± .063	.127 ± .085	.176 ± .096	.153 ± .089	.068 ± .053	.083 ± .063	
spectf	.148 ± .107	.063 ± .046	.122 ± .089	.141 ± .108	.157 ± .113	.130 ± .092	.130 ± .101	
sonar	.080 ± .036	.039 [†] ± .030	.036 ± .025	.073 ± .032	.083 ± .072	.079 ± .031	.077 ± .030	
wine.1	.016 ± .000	.000 ± .000	.000 ± .000	.025 ± .014	.088 ± .062	.023 ± .013	.027 ± .011	
wine.2	.016 ± .013	.023 ± .019	.010 ± .008	.022 ± .016	.026 ± .020	.022 ± .017	.022 ± .016	
wine.3	.045 ± .029	.041 ± .032	.049 ± .030	.040 ± .033	.024 ± .017	.037 ± .031	.030 ± .024	
iris.2	.030 ± .017	.032 [†] ± .025	.038 ± .028	.048 ± .032	.088 ± .057	.048 ± .034	.047 ± .032	
iris.3	.047 ± .027	.000 ± .000	.084 ± .029	.040 ± .029	.035 ± .030	.045 ± .032	.041 ± .028	
Average	.073 ± .080	.055 ± .058	.054 ± .054	.055 ± .060	.055 ± .060	.048 ± .047	.047 ± .047	

Table 7 CAP results, in terms of AE, for binary classifiers trained via MLP.

	Baselines				Our Methods			
	Naïve	ATC	DoC	LEAP _{ACC}	LEAP _{KDE_y}	S-LEAP _{KDE_y}	O-LEAP _{KDE_y}	
poker-hand	.412 ± .086	.326 ± .090	.067 ± .049	.108 ± .078	.087 ± .069	.077 ± .058	.069 [†] ± .057	
connect-4	.170 ± .129	.088 ± .073	.077 ± .059	.070 ± .060	.059 [†] ± .047	.063 ± .052	.057 ± .046	
shuttle	.114 ± .093	.054 ± .053	.050 ± .039	.034 ± .032	.037 ± .035	.032 [†] ± .029	.031 ± .029	
chess	.058 ± .044	.048 ± .037	.039 ± .030	.059 ± .045	.093 ± .056	.042 ± .032	.049 ± .037	
dry-bean	.025 ± .017	.022 ± .018	.018 ± .014	.021 ± .016	.027 ± .019	.021 ± .016	.021 ± .016	
nursery	.001 ± .001	.003 ± .006	.001 ± .001	.003 ± .002	.010 ± .008	.003 ± .002	.005 ± .004	
hand-digits	.008 ± .006	.007 ± .008	.007 [†] ± .006	.008 ± .006	.011 ± .008	.008 [†] ± .006	.007 [†] ± .004	
satellite	.049 ± .043	.036 ± .030	.030 ± .025	.030 ± .025	.032 ± .024	.028 ± .023	.025 ± .020	
digits	.015 ± .013	.015 ± .013	.012 ± .010	.014 ± .012	.023 ± .014	.014 ± .012	.013 ± .010	
page-block	.142 ± .089	.053 ± .044	.039 ± .029	.043 [†] ± .037	.050 ± .037	.041 [†] ± .035	.042 [†] ± .035	
waveform-v1	.031 [†] ± .022	.036 ± .028	.034 ± .024	.030 ± .022	.044 ± .034	.030 ± .022	.030 ± .022	
academic-success	.119 ± .099	.082 ± .061	.075 ± .055	.069 ± .056	.052 ± .042	.050 ± .039	.047 ± .036	
abalone	.055 ± .040	.055 ± .041	.039 ± .030	.058 ± .044	.065 ± .048	.041 [†] ± .032	.047 ± .036	
molecular	.024 ± .018	.034 ± .026	.025 [†] ± .018	.026 ± .019	.031 ± .024	.025 [†] ± .018	.026 ± .018	
image-seg	.025 ± .023	.020 ± .017	.023 ± .020	.019 ± .017	.020 ± .016	.019 ± .018	.017 ± .015	
obesity	.032 ± .024	.036 ± .027	.028 ± .019	.026 ± .020	.060 ± .037	.025 ± .018	.030 ± .021	
cmc	.065 ± .046	.059 ± .045	.057 ± .041	.052 ± .039	.061 ± .047	.044 ± .033	.046 ± .036	
hcv	.045 ± .032	.062 ± .047	.060 ± .040	.054 ± .043	.053 ± .042	.048 ± .036	.052 ± .037	
phishing	.070 ± .060	.054 ± .045	.072 ± .062	.044 ± .035	.045 ± .035	.038 ± .030	.036 ± .026	
yeast	.058 ± .043	.061 ± .046	.077 ± .052	.046 ± .037	.050 ± .038	.047 [†] ± .036	.050 ± .039	
mhr	.085 ± .060	.050 ± .038	.063 ± .042	.051 [†] ± .039	.052 [†] ± .043	.055 [†] ± .037	.055 [†] ± .038	
Average	.076 ± .104	.057 ± .077	.043 ± .043	.041 ± .044	.046 ± .043	.036 ± .036	.036 ± .035	

Table 8 CAP results, in terms of AE, for multiclass classifiers trained via kNN.

C Inference Times of LEAP Methods

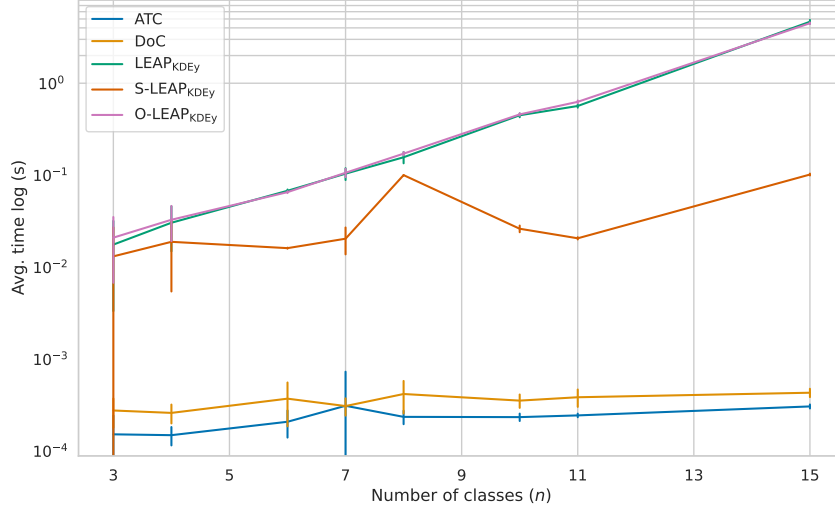


Fig. 4 Average execution times (at inference time) for the multiclass experiments, expressed as a function of the number of classes. Note the log scale along the y-axis.

We also report, in Figure 4, the average inference times (note the log scale on the y-axis) as a function of the number of classes for the multiclass setting. ATC and DoC are markedly faster than the proposed methods, consistently achieving execution times below a millisecond. However, this is not a major concern, as the CAP problem is rarely affected by real-time constraints, and since our methods require, in the worst case we have tested (i.e., for 15 classes), only a few seconds to estimate the classifier’s accuracy. Moreover, one of our methods, S-LEAP, scales linearly with the number of classes, requiring only a tenth of a second in the worst-case scenario. Nevertheless, it is fair to mention that our best-performing method, O-LEAP, scales quadratically with the number of cases, meaning that it is more advisable to use S-LEAP when the number of classes is very large.